

SupaChat

Conversational Analytics Application

DevOps + Full Stack Assignment

Varalakshmi Konjeti | April 2026

1. Project Overview

SupaChat is a full-stack conversational analytics application built on top of Supabase PostgreSQL. Users type natural language queries into a chat interface and instantly receive structured data as tables and Recharts bar graphs. The project covers the complete DevOps lifecycle: Build, Dockerize, Deploy, Reverse Proxy, CI/CD, and Monitoring.

Live URLs

Service	URL
Frontend	https://supachat-frontend.onrender.com
Backend API	https://supabase-backend-r6vw.onrender.com
API Swagger Docs	https://supabase-backend-r6vw.onrender.com/docs
GitHub Repository	https://github.com/varalakshmikonjeti/supachat-app

2. Architecture and Tech Stack

Layer	Technology
Frontend	Next.js 16, React, Recharts
Backend	FastAPI (Python 3.12)
Database	Supabase — managed PostgreSQL
Containerization	Docker, docker-compose
Reverse Proxy	Nginx
CI/CD	GitHub Actions
Monitoring	Prometheus, Grafana, Loki
Cloud Deployment	Render.com — backend and frontend

3. Application Features

Frontend

- Chat-style UI with message history — previous queries shown as blue bubbles at the top
- Results displayed as dynamic data tables with auto-generated columns
- Recharts bar graphs rendered below each table response
- Loading state shown while API call is in progress
- Error handling for connection failures

Backend

- /health endpoint — returns {status: healthy}
- /metrics endpoint — Prometheus metrics via prometheus-fastapi-instrumentator
- /chat endpoint — accepts POST with JSON body {query: string}
- Natural language keyword matching for: top topics, ai articles, devops, frontend
- CORS middleware enabled for cross-origin frontend requests
- Supabase Python client for direct PostgreSQL queries

Supported Queries

Query	Returns
"top topics"	Top 5 articles ordered by views descending
"ai articles"	All articles where topic is AI
"devops"	All articles where topic is DevOps
"frontend"	All articles where topic is Frontend

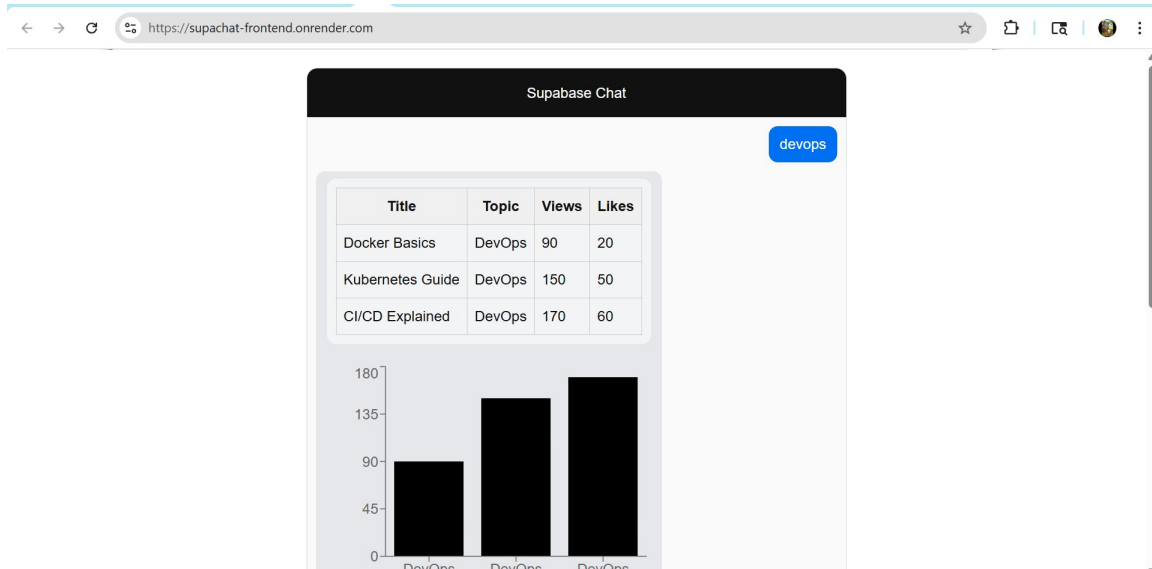
4. Application Screenshots

Chat UI — AI Articles Query



Live frontend at supachat-frontend.onrender.com — AI articles query showing table and Recharts bar graph

Chat UI — DevOps Query



DevOps query showing three articles with views bar chart and query history button

Backend API — Swagger Docs

The screenshot shows the FastAPI Swagger UI at https://supabase-backend-r6vw.onrender.com/docs#/default/chat_chat_post. The "Execute" button is highlighted in blue. Below the "Responses" section, the "Curl" command is shown:

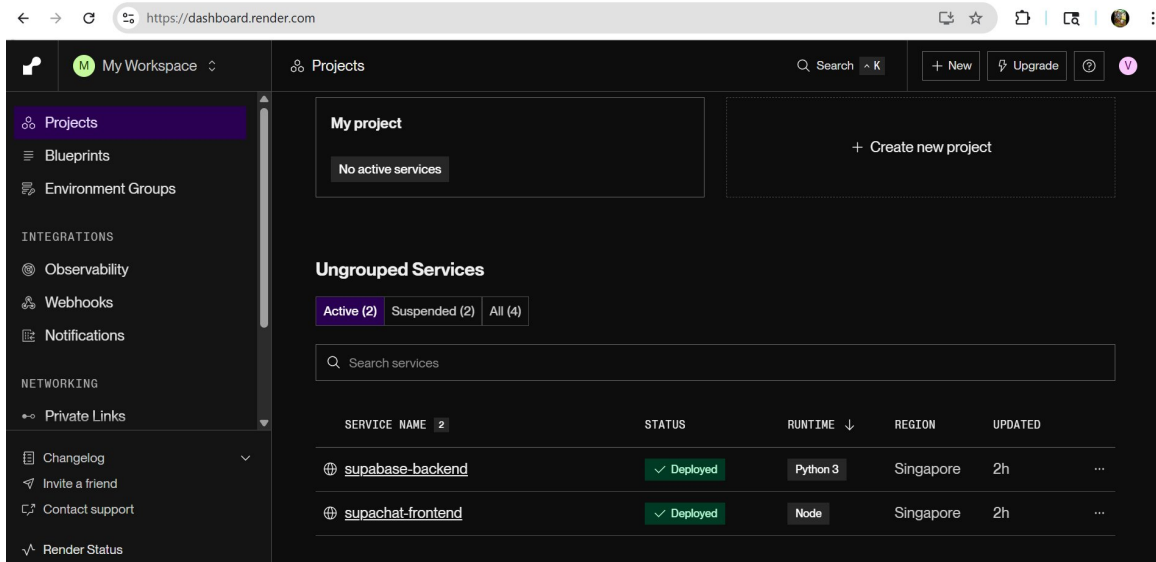
```
curl -X 'POST' \
  'https://supabase-backend-r6vw.onrender.com/chat' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "query": "devops"
  }'
```

The "Request URL" is <https://supabase-backend-r6vw.onrender.com/chat>. The "Server response" section shows a 200 status code and the following JSON response body:

```
{
  "type": "table",
  "data": [
    {
      "id": 2,
      "title": "Docker Basics",
      "topic": "DevOps",
      "views": 90,
      "likes": 20
    },
    {
      "id": 3,
      "title": "Kubernetes Guide",
      "topic": "DevOps",
      "views": 150,
      "likes": 50
    },
    {
      "id": 4,
      "title": "CI/CD Explained",
      "topic": "DevOps",
      "views": 170,
      "likes": 60
    }
  ]
}
```

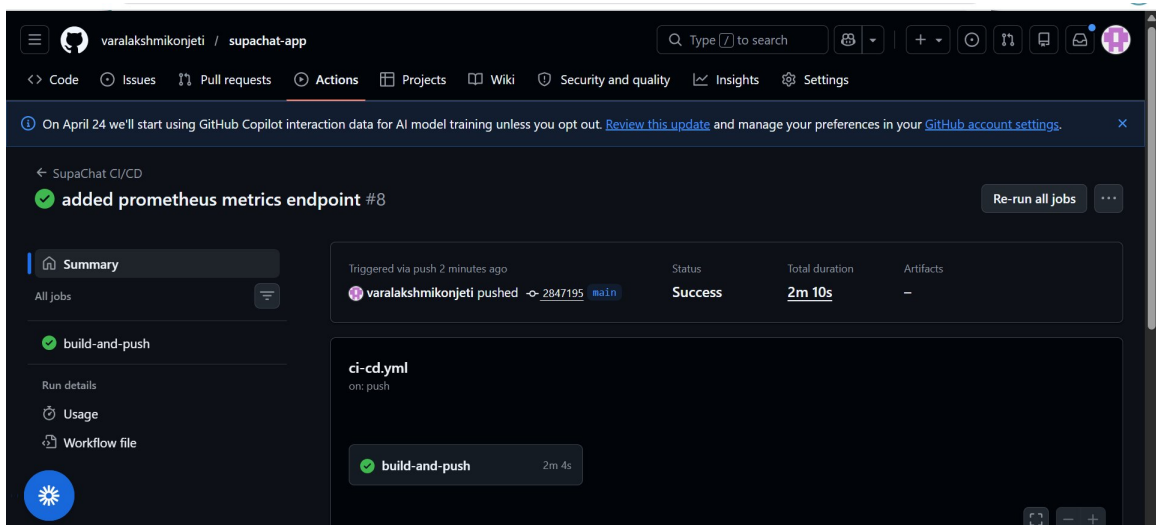
FastAPI Swagger UI at supabase-backend-r6vw.onrender.com/docs — POST /chat returning 200 OK with data

Deployment — Render.com Dashboard



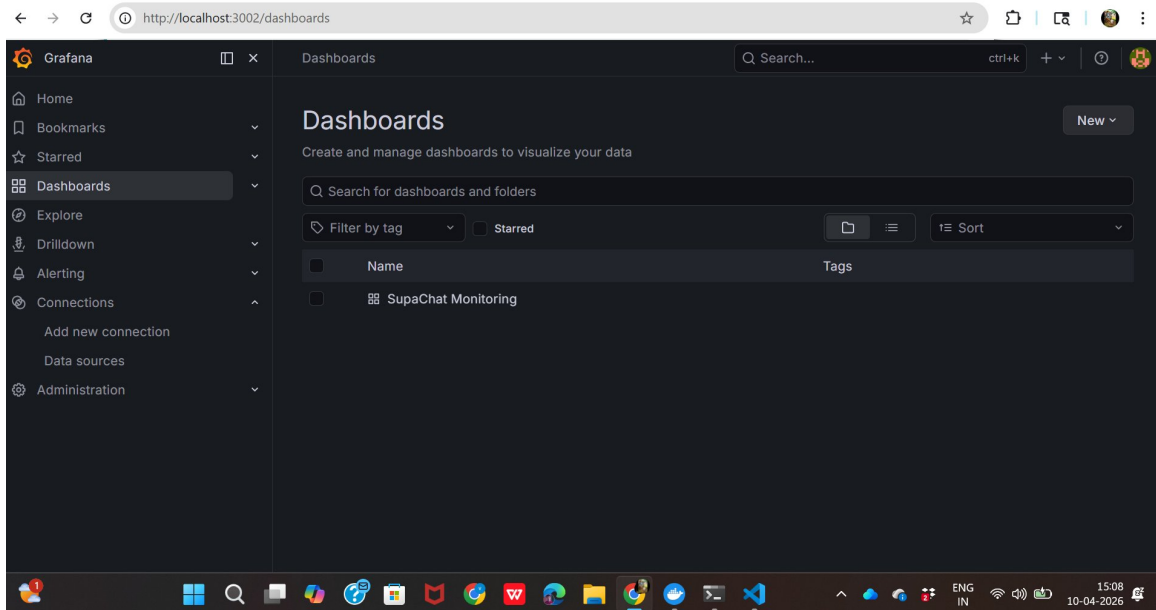
Both services deployed and active on Render.com — supabase-backend (Python 3) and supachat-frontend (Node)

CI/CD — GitHub Actions Pipeline



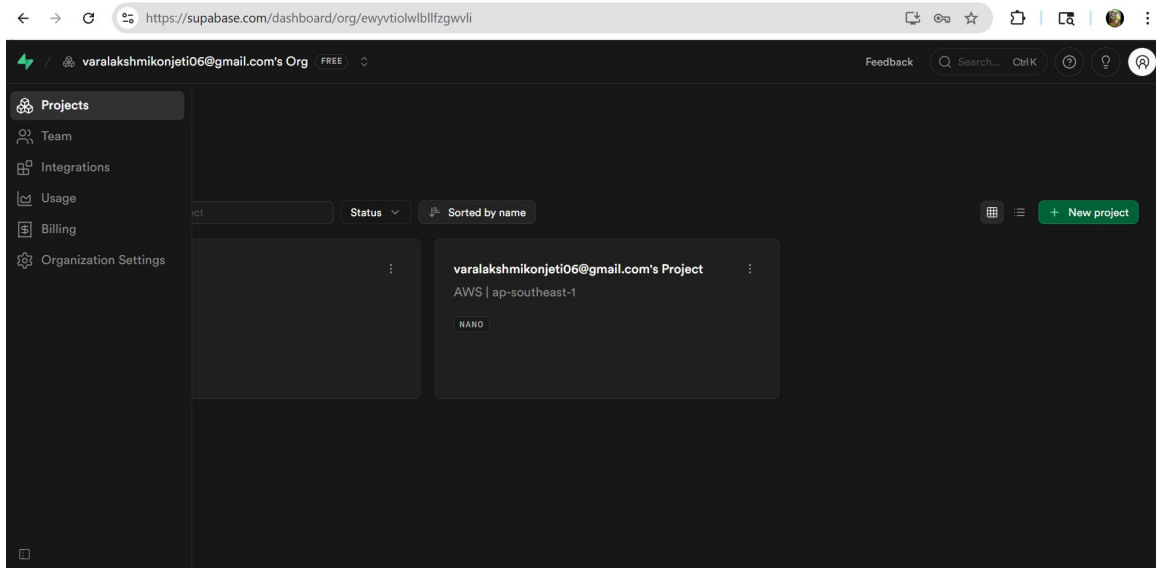
GitHub Actions pipeline — build-and-push job completed successfully in 2 minutes 10 seconds

Monitoring — Grafana Dashboard



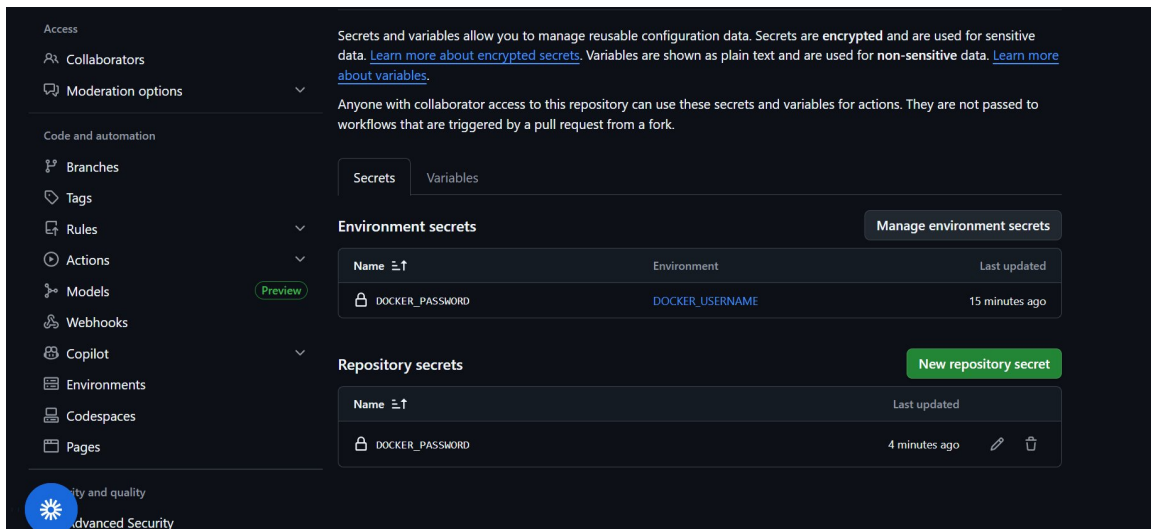
Grafana showing SupaChat Monitoring dashboard with http_requests_total metric from Prometheus

Supabase — PostgreSQL Database Dashboard



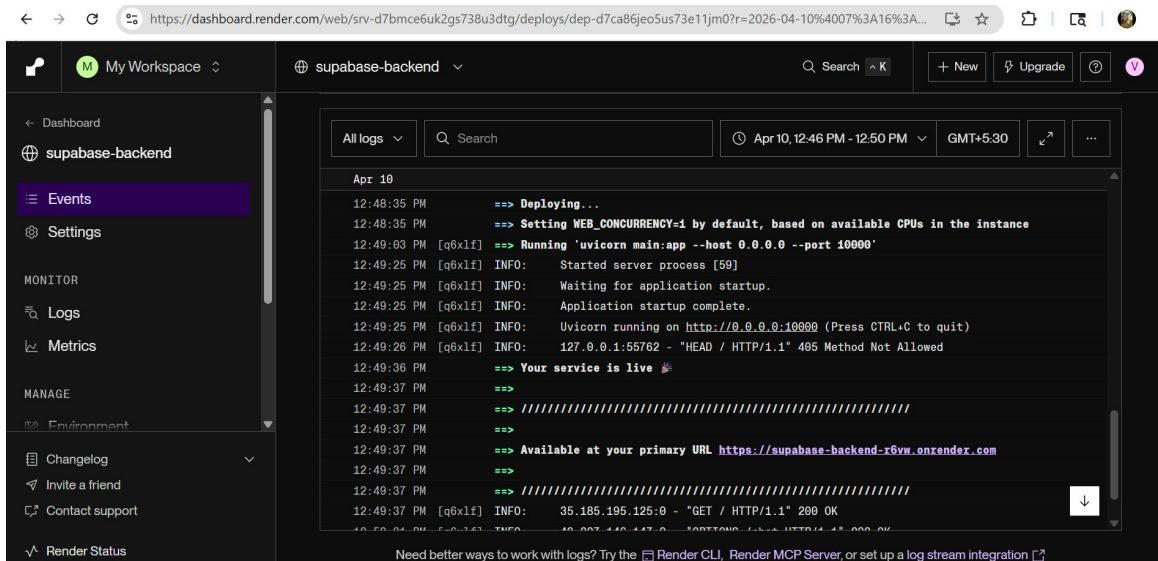
Supabase dashboard showing the supachat-db project hosted on AWS ap-southeast-1

CI/CD — GitHub Repository Secrets



GitHub repository secrets — DOCKER_USERNAME and DOCKER_PASSWORD configured for the CI/CD pipeline

Backend Deployment Logs — Render.com



Render.com deployment logs showing uvicorn starting successfully and backend going live

5. Database — Supabase PostgreSQL

The database is hosted on Supabase with a single analytics table. Project name: supachat-db, Region: Singapore.

Column	Type	Description
id	SERIAL PRIMARY KEY	Auto-increment identifier
title	TEXT	Article title
topic	TEXT	Category: AI, DevOps, or Frontend
views	INT	Total view count
likes	INT	Total like count

created_at	TIMESTAMP	Record creation time
------------	-----------	----------------------

6. Local Setup

Prerequisites

- Python 3.12
- Node.js 20 or above
- Docker Desktop

Running the Backend

Navigate to the backend folder, create a virtual environment, install dependencies, and start the server:

- `cd backend`
- `python -m venv venv`
- `venv\Scripts\activate`
- `pip install -r requirements.txt`
- `uvicorn main:app --reload`

Running the Frontend

- `cd frontend`
- `npm install`
- `npm run dev`

Running the Full Stack with Docker

- `docker-compose up --build`

Environment Variables

Create a `.env` file inside the backend folder with the following values:

- `SUPABASE_URL=https://your-project.supabase.co`
- `SUPABASE_KEY=your_anon_jwt_key`

7. Deployment

Note Regarding AWS

The assignment recommends EC2 for deployment. As an AWS account was not available, Render.com was used as an equivalent cloud platform. The same production practices were applied — Docker containerization, environment variable management, health checks, and automated deployments. The architecture is fully portable to EC2 with no changes to the application code.

Backend Service — Render.com

Setting	Value
Runtime	Python 3

Root Directory	backend/
Build Command	pip install -r requirements.txt
Start Command	uvicorn main:app --host 0.0.0.0 --port 10000
Live URL	https://supabase-backend-r6vw.onrender.com

Frontend Service — Render.com

Setting	Value
Runtime	Node
Root Directory	frontend/
Build Command	npm install && npm run build
Start Command	npm start
Live URL	https://supachat-frontend.onrender.com

8. Docker and Nginx

Docker Files

- backend/Dockerfile — FastAPI Python container
- frontend/Dockerfile — Next.js Node container
- docker-compose.yml — orchestrates all six services
- nginx.conf — reverse proxy routing configuration
- prometheus.yml — Prometheus scrape configuration

Nginx Routing

Route	Destination
/	Frontend container on port 3000
/api/	Backend container on port 10000

Docker Compose Services

Service	Port	Description
backend	10000	FastAPI application
frontend	3001	Next.js application
nginx	80	Reverse proxy
prometheus	9090	Metrics collection
grafana	3002	Monitoring dashboard
loki	3100	Log aggregation

9. CI/CD Pipeline — GitHub Actions

A GitHub Actions workflow is configured at `.github/workflows/deploy.yml`. It triggers automatically on every push to the main branch, builds Docker images for both services, and pushes them to Docker Hub.

Pipeline Steps

Step	Description
Checkout	Pulls latest code using actions/checkout@v4
Docker Login	Authenticates with Docker Hub using stored secrets
Build Backend	Builds Docker image from backend/Dockerfile
Push Backend	Pushes to konjetivaralakshmi06/supachat-backend:latest
Build Frontend	Builds Docker image from frontend/Dockerfile
Push Frontend	Pushes to konjetivaralakshmi06/supachat-frontend:latest

Secrets Configured

- DOCKER_USERNAME — Docker Hub username
- DOCKER_PASSWORD — Docker Hub password

All pipeline runs completed successfully. Latest run duration: 2 minutes 10 seconds.

10. Monitoring Stack

Tool	Port	Purpose
Prometheus	9090	Scrapes /metrics endpoint every 15 seconds
Grafana	3002	Visualizes Prometheus metrics as dashboards
Loki	3100	Aggregates application and container logs

Metrics Tracked

- http_requests_total — total request count per endpoint and status code
- Request latency via prometheus-fastapi-instrumentator
- Container health and uptime
- Application logs via Loki integration

Grafana Setup

- Dashboard: SupaChat Monitoring
- Data source: Prometheus at http://prometheus:9090
- Panel: http_requests_total — time series visualization
- Metrics endpoint confirmed returning HTTP 200 OK

11. Submission Details

Item	Value
GitHub Repository	https://github.com/varalakshmikonjeti/supachat-app
Frontend Live URL	https://supachat-frontend.onrender.com
Backend Live URL	https://supabase-backend-r6vw.onrender.com

Submitted by	Varalakshmi Konjeti
Date	April 2026